UniCRM Project Report

A simple overview of the system architecture, features, and design patterns

1. Introduction

UniCRM is a management system built for universities. It is written in Java and follows Object-Oriented Programming (OOP) principles. The project is split into separate layers, which keeps the code clean, easy to understand, and simple to change or maintain.

2. Project Architecture

The system uses a layered structure. Each layer has one specific job:

- **Domain Layer:** Contains the basic data models and objects, such as `User`, `Student`, `Teacher`, `Course`, and `ResearchPaper`.
- **Repository Layer:** Responsible for saving and loading data using files (like `users.dat` or `courses.dat`).
- **Service Layer:** Holds the main business rules and logic of how the university works (e.g., how to calculate grades or register for a course).
- **Command Layer:** Turns user actions into clear tasks (commands) that call the right services.
- **Session & State:** Keeps track of the currently logged-in user and their active session.

3. Main Features

3.1 Academic Management

This part manages classes, students, and grades:

- **Course Registration:** Students can sign up for available courses. Managers review these requests to approve or reject them.
- **Grading & Transcripts:** Teachers put in grades for exams and attestations. The system uses this data to update student transcripts and track how many courses a student has failed.
- **Teacher Assignment:** Managers can assign specific courses to teachers.

3.2 Communication & Tech Support

This part helps university members interact and resolve technical problems:

- **Direct Messaging:** Employees can send and receive text messages.

- **Complaints:** Teachers can file formal complaints against students and set an urgency level (Low, Medium, High).
- **Tech Support Tickets:** Users can create support requests. Tech Support Specialists view new requests, assign themselves as executors, and change ticket statuses (like ACCEPTED or DONE).

3.3 Research Module

This part is for university members who do scientific research:

- **Publishing Papers:** Researchers can publish research papers in university journals and generate citations in formats like Plain Text or BibTeX.
- **Subscriptions:** Users can subscribe to university journals to get notified when new work is published.
- **Research Metrics:** The system tracks metrics like citations and the h-index to find top researchers.

4. Design Patterns Used

The project uses several classic design patterns to solve common programming problems cleanly:

Command Pattern

Every user action is turned into a separate class (like `PutMarkCommand` or `RegisterForCourseCommand`). This makes it very easy to add new features or menu items without changing the existing codebase.

Decorator Pattern

Used for the research feature via `ResearcherDecorator`. Instead of making complex subclasses like StudentResearcher or TeacherResearcher, the system wraps a standard `User` object dynamically to give them research abilities at runtime.

Singleton Pattern

Used in `UserSession`. It guarantees that only one session instance exists at any time. This provides a single, reliable place to check who is currently logged into the app.

Repository Pattern

Classes like `UserRepository` handle saving and loading data. The rest of the application does not need to know how or where the data is stored (such as in files), making it easy to swap file storage for a database later.

Observer Pattern

Used for journal subscriptions. When a researcher publishes a new paper in a `UniversityJournal`, all users subscribed to that journal automatically get an update.